

AtlasSplit | 面试问题全集 · 求职精简版

自然语言表格 Agent · 2026-09-08

严格按《求职指导_MindTrip_AtlasSplit_项目指标与面试准备方案(1)》精简 · P0 必须掌握 / P1 高频追问

这版怎么用

- 严格围绕求职指导中的 Planning、AllocationPlan/DSL、受控代码执行、Repair、Task Success、安全、Audit Replay 与 Excel/CSV 工程边界；把字段级、Docker 参数级拆题全部合并。
- 先把 P0 全部讲顺，再补 P1；P1 是面试官从 P0 向下追问 1-3 层时最常出现的问题。
- 不再为每个字段、每个 Docker 参数、每个迁移场景单独拆题；目标是能面试，不是背百科。

0、先背这张证据卡 架构可以讲，结果数字必须等实测

证据层	当前已确认	面试边界
执行架构	Data Profiling -> Planning -> Validate -> Execute -> Verify；复杂路径加入 Pandas Code -> AST Audit -> Docker Sandbox -> Verify -> Repair/Retry -> Audit Replay。	这是实现事实，不等于已有准确率/成功率/安全率。
规划/正确性指标	Rule Parsing Accuracy、Schema Valid Rate、First-pass / Final Task Success、Repair Recovery Rate。	当前没有正式固定业务集上的确认百分比。
安全指标	Dangerous Code Detection Recall + False Positive Rate；覆盖危险 import、命令、网络、敏感文件、资源滥用、eval/exec/动态加载等。	当前尚无正式 adversarial benchmark 与 FP/FN 数字；100-200 条只是建议规模。
规则覆盖	已确认资料中出现等额、权重、封顶/保底、阶梯区间、跨表聚合等。	简历若写“6 类规则”，必须从真实实现固定完整六类名称与样例，不能现场编。

当前最重要的待补项

- 固定业务任务集大小与任务分布。
- Rule Parsing Accuracy / Schema Valid Rate / First-pass & Final Task Success。
- Repair Recovery Rate + 至少 2 条 Repair 成功、1 条仍失败的真实案例。
- Dangerous Code Detection Recall / False Positive Rate + FP/FN 样例。

一、评测与证据链

P0 1. AtlasSplit 的每个指标都必须先说清哪四件事？——测试集、指标定义、Baseline、失败案例。

P0 2. Rule Parsing Accuracy、Task Success、Safety Recall 各自应该在哪类固定测试集上评？

P0 3. Ground Truth 为什么要包含 expected AllocationPlan / deterministic checks，而不能只让 LLM 自评？

P0 4. Task Success 为什么必须是可重复判定，而不是“看起来结果对了”？

P1 5. 当前哪些百分比还没有正式实测，为什么不能拿参考目标区间或建议样本量代替？ [待补正式 Benchmark]

P1 6. 正常业务任务和安全 adversarial case 为什么必须分开统计？

P1 7. AtlasSplit 至少要准备哪些正常失败案例和安全 FP/FN 案例？ [待补真实案例]

二、LLM 与确定性执行

P0 8. 为什么不让 LLM 直接操作 Pandas？

P0 9. LLM 适合负责什么，确定性执行引擎适合负责什么？

P0 10. 为什么要把 Planning 与 Execution 分开？这种分层降低了哪些不可控行为？

P1 11. 如果 LLM 已经能生成代码，为什么仍然需要结构化 Plan 和执行前校验？

三、AllocationPlan / DSL 与规划准确率

P0 12. AllocationPlan 本质是什么？为什么它可以同时理解为 Contract 和 DSL？

P0 13. AllocationPlan 位于哪两个模块之间，核心字段包括哪些？

P0 14. operation、columns、filters、group_by、allocation_rule、output 分别表达什么语义？

P0 15. Rule Parsing Accuracy 怎么定义？什么情况下自然语言需求才算“解析正确”？

P0 16. Schema Valid Rate 证明什么？为什么它不能替代业务正确率？

P1 17. 如果某个 Plan 能通过 Pydantic/Schema，但列选错、过滤条件错，应该算成功吗？为什么？

四、Fixed DSL 与动态 Code Generation

- P0 18. 已经有 AllocationPlan，为什么复杂任务仍可能需要动态 Code Generation？
- P0 19. 固定 DSL 与动态代码生成各有什么优缺点？Hybrid 方案怎么路由？
- P1 20. 哪些普通任务应该优先走结构化 Plan，哪些复杂任务才允许进入代码生成？
- P1 21. 为什么越靠近动态代码执行，越需要更强的安全与验证机制？

五、代码安全：AST Audit + Docker Sandbox

- P0 22. LLM 生成的 Pandas Code 为什么不能直接执行？主要风险有哪些？
- P0 23. 有 Docker 了为什么还需要 AST Audit？两层分别解决什么问题？
- P0 24. 为什么 AST + Sandbox 属于 Defense in Depth，而不能宣称“100% 安全”？
- P0 25. 危险 import、系统命令、网络访问、敏感文件、无限循环/资源滥用、eval/exec/动态加载分别应该怎样限制？
- P0 26. Docker Sandbox 还要控制哪些运行期边界：network、文件权限、CPU/memory、timeout、process capability？
- P1 27. 为什么能静态拒绝的高风险代码应该尽量在 AST 阶段提前拒绝？
- P1 28. AST 规则的误报和漏报分别会带来什么问题？

六、Repair / Retry

- P0 29. Repair/Retry 为什么有用？什么错误适合 Repair，什么错误应该直接拒绝？
- P0 30. 怎样通过 max_retry、错误分类、retryable/non-retryable 避免死循环？
- P0 31. 为什么要同时统计 First-pass Success 和 Final Success？

P0 32. Repair Recovery Rate 怎么计算，它真正证明的是什么？

P1 33. 要证明 Repair 真有价值，至少保留哪些“修复成功/仍失败”的真实案例？ [待补真实案例]

七、End-to-End Task Success

P0 34. AtlasSplit 的 Task Success 为什么定义为 Plan Valid + Execution Success + Output Verify + Business Constraints Pass?

P0 35. Plan Valid、Execution Success、Output Verify、Business Constraints Pass 各自是什么意思？

P0 36. 为什么“代码执行没有报错”远远不等于任务成功？

P1 37. 首次成功率和 Repair 后最终成功率应该如何放在同一张评测报告里比较？ [待补实测]

八、安全指标

P0 38. Dangerous Code Detection Recall 是什么？为什么安全场景更怕 False Negative?

P0 39. 为什么还要同时看 False Positive Rate，避免安全策略把正常任务全误杀？

P0 40. 安全对抗 case 应该覆盖哪些类别，怎样构造才可复现？

P0 41. 为什么不能写“100% 安全”，更可信的简历表达应该是什么结构？

P1 42. 正常任务与安全 case 的分母、Ground Truth、FP/FN 样例应该怎样保存？ [待补安全 Benchmark]

九、Audit Replay 与公平模型比较

P0 43. Audit Replay 有什么价值？如何用于 Debug、事故追踪、回归评测和失败案例分析？

P0 44. Planning、Code Generation、AST Audit、Execution、Output 分别需要记录哪些信息？

P0 45. 如果换模型，怎样固定 Dataset、Prompt、Tools、Temperature 和 Eval 规则做公平比较？

P1 46. 模型比较应该同时看哪些指标：Task Success、First-pass、Retry Rate、Latency、Token/Cost?

十、Excel / CSV 工程边界

P0 47. Excel / CSV 为什么不能只测试 happy path?

P0 48. UTF-8/BOM、类型推断、空值、日期、公式、合并单元格、大文件内存这些边界应该怎样分类准备?

P1 49. 遇到公式、合并单元格或类型推断异常时，系统应该拒绝、澄清还是降级处理?

P1 50. 大文件为什么需要关注内存、读取方式和执行超时，而不仅是 Pandas 逻辑正确性?

最后 30 秒自检

真正要背熟的 5 句话

- LLM 负责意图理解/规划，确定性 Pandas/执行器负责可重复执行与验证。
- AllocationPlan 是 LLM 与执行引擎之间的结构化 Contract/DSL。
- 动态代码先 AST 静态审计，再 Docker 运行期隔离，再 Verify；这是 Defense in Depth，不是“100% 安全”。
- Task Success = Plan Valid + Execution Success + Output Verify + Business Constraints Pass。
- Repair 的价值必须用 First-pass -> Final Success 和 Repair Recovery Rate 证明；安全必须用 Detection Recall + FPR + FP/FN 证明。

AtlasSplit 面试题逐题回答

50 道原题 · 第一人称口述稿 · 2026 年 9 月 8 日

原题完整保留在本文件前 5 页，以下按原编号逐题作答。回答区分已实现机制、实际排障经历与尚未实测的指标。原题中的通用 DSL 字段、六类规则及完整回放描述，不应直接当作当前实现事实；相关回答已明确说明。

建议先理解每题中的判断和例子，再换成自己的表达。尚未亲自参与或无法复现的环节，不宜背成个人经历。

评测与证据链

P0 1. AtlasSplit 的每个指标都必须先说清哪四件事？——测试集、指标定义、Baseline、失败案例。

我会先交代测试集是什么、怎么算成功、跟谁比较，以及失败在哪里。比如分摊任务，测试集要固定输入文件和自然语言要求，成功要由金额守恒、对象集合和业务规则共同判定，Baseline 要明确是初版规划器还是另一个模型，最后给出没处理好的例子。做这个项目以后，我对数字反而更谨慎了：单元测试通过能证明某些工程行为，但不能直接换算成业务成功率。没有这四个前提，一个百分比其实很难说明问题。

P0 2. Rule Parsing Accuracy、Task Success、Safety Recall 各自应该在哪儿类固定测试集上评？

这三个指标不能用同一个分母。解析准确率应该在带标准意图和预期计划的业务集上测，重点看模型有没有理解对；Task Success 要在可以真正执行、并且有确定性验收条件的端到端任务集上测；Safety Recall 则在明确标注为危险的代码或行为样本上测。正常代码还要单独保留，用来算误报率。我现在不会拿安全样本拦截了多少条，去证明正常表格任务做得好，也不会拿固定规则测试去证明模型理解准确。

P0 3. Ground Truth 为什么要包含 expected AllocationPlan / deterministic checks，而不能只让 LLM 自评？

因为生成计划和评价计划如果都交给同一个模型，很容易把同一种误解重复两遍。用户说按部门分摊，模型选成按员工分摊，结果可能也能写出一段很合理的解释。我更愿意把标准答案拆成两层：一层是预期计划的关键语义，比如对象列、方法、总额来源；另一层是运行后的确定性检查，比如对象有没有漏、权重比例和金额总和对不对。LLM 可以辅助解释差异，但最后是否通过，不能靠它自己点头。

P0 4. Task Success 为什么必须是可重复判定，而不是“看起来结果对了”？

我希望同一份输入、同一份已批准计划，在相同执行条件下能得出同一个验收结论，而不是今天看着对、明天换个人又觉得不对。比如一千元按一比三分给两个部门，不能只检查生成了 Excel，还要检查对象是那两个部门、金额是二百五和七百五、总额是一千。这里要区分模型重新规划和确定性执行：模型未必每次生成一样的计划，所以可重复判定首先要固定已持久化的计划，而不是假设整个 LLM 链路天然确定。

P1 5. 当前哪些百分比还没有正式实测，为什么不能拿参考目标区间或建议样本量代替？[待补正式 Benchmark]

这份题单里的规则解析准确率、首次和最终任务成功率、Repair 恢复率，以及危险代码召回率和误报率，我目前没有一套已经确认的正式业务 Benchmark 百分比可以直接报。仓库有测试和历史检查记录，但它们不自动等于这些指标。建议做一百条样本，不代表已经做完一百条；目标达到九成，也不代表实际达到了。我的面试表达会先讲已经实现并验证的机制，再说明哪些数字尚未建立统一口径，不用目标值包装结果。

P1 6. 正常业务任务和安全 adversarial case 为什么必须分开统计？

正常业务任务是在问“该做的有没有做对”，安全对抗是在问“不该做的有没有被挡住”，两者方向不同。假设系统把所有请求都拒绝，危险样本可能全被拦住，但这个产品也就没法用了。所以我会把业务成功率、安全召回率和正常任务误报率分开看，再结合起来判断策略是否合理。这个区别对我的项目很实际，因为 AST 白名单收得太紧，确实可能挡住合法的 Pandas 写法；放得太松，又会扩大执行风险。

P1 7. AtlasSplit 至少要准备哪些正常失败案例和安全 FP/FN 案例？[待补真实案例]

我会准备三类正常失败：需求有歧义但系统选错列、数据本身不满足规则、执行环境出了问题。安全方面则各准备一条合法代码被拦的误报和一条危险行为未被预期层拦住的漏报，并保留输入、规则版本、判定理由和最终处置。这里我会把真实经历和待补材料分开讲：项目里实际排查过审批失效、文件锁并发和测试目录误写，但它们是工程故障，不是安全分类器的 FP/FN 样本。我不会把这几件事硬凑成已经完成的安全评测。

LLM 与确定性执行

P0 8. 为什么不让 LLM 直接操作 Pandas？

不是说 LLM 不能写 Pandas，而是不能让它写完就直接对用户文件执行。这个项目涉及金额分摊，模型哪怕只选错一列，程序也可能正常结束，结果却已经错了；如果生成代码还能访问文件和网络，风险就更大了。所以常规任务我先让模型输出结构化计划，再交给确定性执行器。旧的代码生成路径仍然存在，但需要显式进入受控流程。对我来说，模型负责提出方案，系统负责决定这个方案能不能执行。

P0 9. LLM 适合负责什么，确定性执行引擎适合负责什么？

LLM 比较适合把自然语言转换成候选业务意图，比如理解“按各部门权重分摊”，识别可能对应的列，或者把不确定的地方整理成澄清问题。确定性代码负责字段校验、数值计算、文件边界、权限判断和结果验收。后面做记忆治理时我也沿用了这个原则：记忆可以帮助模型理解，但不能覆盖当前输入、用户确认和执行规则。我不希望模型因为曾经见过一个相似任务，就自行取得修改这次数据的权限。

P0 10. 为什么要把 Planning 与 Execution 分开？这种分层降低了哪些不可控行为？

最直接的原因是用户要有机会在产生副作用之前看懂系统准备做什么。分开以后，计划可以持久化、预览、审批，也能绑定输入文件和配置。项目里一个很典型的风险是，用户看到的是旧计划，但点击批准时服务器已经换了上下文。为了解决它，批准请求会携带用户实际看到的摘要，服务器在锁内比较，并在执行前重新检查有效性。分层不只让代码清楚，还让“我批准的到底是什么”有了可以核对的答案。

P1 11. 如果 LLM 已经能生成代码，为什么仍然需要结构化 Plan 和执行前校验？

代码能生成，只说明模型给出了一个程序，不说明程序满足需求，也不说明它被授权执行。结构化 Plan 把对象列、总额、权重和输出目标摆在明面上，能在执行前发现未知字段、缺少权重列、非有限金额等问题，还能向用户展示影响范围。我遇到过的一类隐患是，前端按钮看起来经过确认，后端却没有绑定用户当时看到的版本。计划和审批上下文就是为了解决这类问题，不能用“模型代码写得不错”来替代。

AllocationPlan 与规划准确率

P0 12. AllocationPlan 本质是什么？为什么它可以同时理解为 Contract 和 DSL？

我把 AllocationPlan 理解成一个范围比较小的业务语言：它能描述等额或权重分摊，但不能随便执行表达式。它是 DSL，是因为它用固定字段表达特定领域的操作；它又是 Contract，是因为规划器和执行器都必须遵守这套字段及约束。这个设计的价值不是让 JSON 看起来整齐，而是把模型的自由度收缩到系统真正支持的能力里。超出能力的请求应该澄清或明确拒绝，不能塞进一个看似合法的字段里继续跑。

P0 13. AllocationPlan 位于哪两个模块之间，核心字段包括哪些？

它在规划器和确定性执行器之间。按当前代码，核心字段有 source_sheet、entity_column、method、total 或 total_column、total_aggregation、weight_column，以及精度和输出工作表等配置；解释和歧义也会单独表达。这里我会主动纠正题单的一处概括：当前 AllocationPlan 不是一个包含任意 operation、filters 和 group_by 的大而全模型。通用表格步骤放在另一个 TablePlan 里。把概念图里的字段当成实际代码字段去背，面试官追到实现时很容易露出矛盾。

P0 14. operation、columns、filters、group_by、allocation_rule、output 分别表达什么语义？

从概念上说，operation 是做什么操作，columns 是涉及哪些列，filters 是保留哪些记录，group_by 是按什么维度归并，allocation_rule 是怎么分配，output 是结果写成什么样。但落到这个项目，我不会说这六个名字都是 AllocationPlan 的原字段。当前通用流程用 TablePlan.steps 表达 filter、group、sort、select、limit 和 allocate；分摊步骤再使用 AllocationPlan。这样解释既能回答这些概念的语义，也能说明真实实现没有为了覆盖概念图而堆一个万能结构。

P0 15. Rule Parsing Accuracy 怎么定义？什么情况下自然语言需求才算“解析正确”？

我会把它定义为：在固定、可判定的业务样本中，计划关键语义与标准意图一致的比例。比较时不应该苛求解释文字一模一样，而应该看对象、总额来源、方法、筛选或聚合语义是否一致。例如用户要求按销售额加权，计划却用了利润列，即使两个字段都是数值列，也应判为解析错误。对于本来就有歧义的题，标准结果可以是提出正确的澄清问题，而不是强行生成一个可执行计划。这种口径需要在跑测试前就定好。

P0 16. Schema Valid Rate 证明什么？为什么它不能替代业务正确率？

Schema Valid Rate 说明输出在结构和约束上能被系统接受，比如字段类型正确、没有未知字段、加权分摊提供了权重列。它更像是“能不能进入下一步”的指标，而不是“业务有没有理解对”。我在项目里确实补过拒绝未知字段和 NaN、Infinity 这类校验，但这些只能拦住一部分错误。模型完全可能交出一个格式很漂亮、业务含义却错了的计划，所以 Schema 通过后，还需要结合数据检查、业务验证和必要的人工审阅。

P1 17. 如果某个 Plan 能通过 Pydantic/Schema，但列选错、过滤条件错，应该算成功吗？为什么？

不算成功。列选错或者过滤条件错，本质上就是执行了另一个任务。最危险的地方在于，这类错误往往不会报异常，生成的表格甚至很整齐。我会把它记到语义解析或业务约束失败，而不是因为 Pydantic 通过就放行。前端做字段映射辅助也是出于这个原因：把列名、类型和样例摆出来，让用户明确选择，并将选择带入新任务草稿；它不会悄悄改掉已经批准的计划，也不能替代后端校验。

固定 DSL 与动态代码

P0 18. 已经有 AllocationPlan，为什么复杂任务仍可能需要动态 Code Generation？

固定 DSL 只能覆盖事先建模的操作，复杂的清洗、特殊计算或不规则转换，确实可能超出它的表达能力。不过“可能需要代码”不等于“遇到不会的需求就自动生成代码”。我现在更倾向于先判断这个需求是否经常出现、能否用有限的表格步骤表达，只有确实需要且经过明确授权，才考虑代码路径。题单提到封顶、保底和跨表规则，我不会据此宣称当前主路径已经完整支持它们；主路径稳定覆盖的是等额和权重分摊。

P0 19. 固定 DSL 与动态代码生成各有什么优缺点？Hybrid 方案怎么路由？

固定 DSL 的优势是可校验、可预览、可回放，缺点是表达范围有限；动态代码灵活，但安全、测试和结果解释的成本明显更高。理想的 Hybrid 路由应该先走能力判断：已支持的操作进入结构化计划，有歧义就澄清，超出范围的请求再讨论受控代码。当前项目实际做的是结构化路径优先，旧代码生成入口显式保留，不是一个已经完成的全自动智能路由器。我觉得把这条边界讲清楚，比说自己支持任意表格任务更可信。

P1 20. 哪些普通任务应该优先走结构化 Plan，哪些复杂任务才允许进入代码生成？

像明确总额的等额分摊、按已知数值列加权，应该优先走 AllocationPlan；有限的筛选、聚合、排序可以考虑已有的 TablePlan 能力。特殊清洗、多步骤自定义计算这类确实超出固定操作集合的需求，才有讨论代码生成的必要，而且要先确定验收标准。当前 TablePlan 的代码存在，不等于所有 Web 和 HTTP 流程都已经接通。我的做法是把“底层具备能力”和“用户入口完整可用”分开说明，避免演示时才发现支持范围被说大了。

P1 21. 为什么越靠近动态代码执行，越需要更强的安全与验证机制？

因为越接近代码执行，模型输出就越可能从一段描述变成真实的文件写入、资源消耗，甚至越权访问。一个错误的计划可以在预览阶段撤回，已经执行的代码却可能留下难以恢复的副作用。所以这里需要层层缩小权限，并验证实际结果。项目里我也逐渐意识到，只校验代码内容还不够：批准是否过期、输入有没有变化、旧执行代次是否仍有权限，都必须检查。安全边界不仅在沙箱里，也在“谁现在有权启动它”这一层。

AST 与运行期隔离

P0 22. LLM 生成的 Pandas Code 为什么不能直接执行？主要风险有哪些？

主要有两类风险。一类是业务错误，比如列理解错、遗漏记录、金额计算不守恒；另一类是执行风险，比如读到不该读的文件、发网络请求、启动进程或耗尽资源。LLM 还可能把表格里的文本误当作指令，所以输入数据也不能被当成可信控制信息。我不会因为提示词写了“只做数据分析”，就认为这些风险消失。常规任务尽量不用动态代码，必须走代码路径时，再做静态审计、运行期隔离和独立结果检查。

P0 23. 有 Docker 了为什么还需要 AST Audit？两层分别解决什么问题？

AST 审计解决的是“这段代码里有没有已知不允许的结构”，Docker 解决的是“代码真的跑起来以后能接触到什么、能消耗多少资源”。静态层可以提前拒绝危险导入、动态执行和反射访问，没必要等进容器才发现。容器层则约束静态分析看不全的运行行为，例如资源消耗和文件访问范围。这两层不是相互替代的。项目还有 subprocess 后端，但它没有 Docker 同等级的网络和文件系统隔离，我不会把二者混称为同一种沙箱。

P0 24. 为什么 AST + Sandbox 属于 Defense in Depth，而不能宣称“100% 安全”？

因为每一层都有边界。AST 白名单可能漏掉间接行为，也可能误拦正常代码；Docker 的效果依赖实际启动配置、挂载范围和宿主环境，不能只看项目里有个 Docker 类就认为安全。我的表达会是“做了静态白名单和运行期隔离，降低特定风险”，而不是“百分之百安全”。有一个细节我会特别说明：本地子进程的超时和资源限制，不等于完整隔离。把后端差异和未验证边界说清楚，本身就是安全工程的一部分。

P0 25. 危险 import、系统命令、网络访问、敏感文件、无限循环/资源滥用、eval/exec/动态加载 分别应该怎样限制？

我会按风险分层处理：危险导入、系统命令、eval/exec 和动态反射，优先在 AST 阶段拒绝；网络和敏感文件还必须靠执行环境限制，不能只靠查函数名；无限循环和资源滥用主要靠超时、内存、CPU 和进程数边界。当前 AST 有导入白名单、危险内置函数和双下划线访问等检查，Docker 后端也有网络和资源配置。但这不是一个完整的 Python 行为证明器，对第三方库内部行为仍要保持保守，不能因表面没有危险关键字就放松隔离。

P0 26. Docker Sandbox 还要控制哪些运行期边界：network、文件权限、CPU/memory、timeout、process capability?

我核对过当前 Docker 调用，已经设置了只读根文件系统、临时目录、网络开关、内存、CPU、进程数和等待超时；外部原始输入按只读方式挂载，工作区则需要允许输出。这里的网络限制受配置控制，不该被描述成所有部署都绝对断网。至于显式降权用户、cap_drop 和 no-new-privileges，当前这段启动调用里没有完整体现，我不会说已经做全。面试时我会讲清已有控制，再把这些列为需要进一步核对和补齐的部署加固项。

P1 27. 为什么能静态拒绝的高风险代码应该尽量在 AST 阶段提前拒绝?

提前拒绝最直接的好处是，不让明显不该执行的代码走到真正产生副作用的阶段，也减少容器启动和模型修复的成本。另一个好处是错误更可解释：告诉用户某个导入不在允许范围，比执行后只得到一个模糊失败更容易定位。不过静态规则要稳定、可测试，不能为了拦住一个例子就随手加一个字符串判断。我的原则是能在更早、权限更低的阶段确定拒绝的，就在那里拒绝，但不要把这种提前检查误说成完整的运行期安全保证。

P1 28. AST 规则的误报和漏报分别会带来什么问题?

误报会让合法任务做不了，用户会觉得系统能力弱，甚至开始想办法绕开安全开关；漏报则可能让危险行为进入执行层，后果通常更严重。所以我不会单独追求“拦截越多越好”。应该看清楚误报挡住了哪一种正常写法，漏报又突破了哪一层边界，再决定是改 AST 规则、收紧容器权限，还是减少可执行能力。目前没有确认的正式 FP/FN 比例，我可以解释这个判断框架，但不能报一个看起来很漂亮的数字。

Repair 与 Retry

P0 29. Repair/Retry 为什么有用？什么错误适合 Repair，什么错误应该直接拒绝?

Repair 适合修正局部、可定位，而且不会改变授权范围的错误，比如结构化输出格式不合法，或允许范围内的字段表达错误；Retry 更适合限流、短暂网络故障这类暂时性问题。危险操作、权限不足、输入语义不明确，不应该靠多试几次解决。项目中我还处理过后台任务把 TypeError、AssertionError 当成可重试故障的问题，这类更像程序缺陷，应进入降级状态等待明确处理。反复调用模型不但修不好，还可能持续消耗费用。

P0 30. 怎样通过 max_retry、错误分类、retryable/non-retryable 避免死循环?

我不会只设一个 max_retry 就认为循环有界了，因为外层工作流、模型客户端和修复循环可能各重试几次，叠起来远超预期。现在的方向是统一错误分类，并在活跃规划轮次共享调用预算和截止时间；API 和 CLI 的预算会在调用 SDK 前持久化预留，重启后不能凭空恢复额度。人工澄清的等待不应消耗活跃计算时限。对于结果不明的调用，不随便退还额度。这样约束的是实际发生了多少次调用，而不只是某一层循环执行了几次。

P0 31. 为什么要同时统计 First-pass Success 和 Final Success?

因为最终成功率会掩盖系统第一次到底有多不稳定。假设两个方案最终都能完成任务，一个一次就对，另一个平均要修三次，它们的延迟、费用和用户体验完全不同。首次成功率更能反映初始规划质量，最终成功率说明有限修复之后的可恢复能力，两者放在一起才有意义。做预算持久化时我也有这种感受：不能只盯最后有没有成功，还要看过程中付出了多少调用，以及失败后是否还在无意义地继续尝试。

P0 32. Repair Recovery Rate 怎么计算，它真正证明的是什么？

我会先固定口径：在进入允许修复流程的首次失败任务中，最终通过全部验收的任务数，除以这批允许修复任务的总数。被安全规则直接拒绝、用户取消或者根本不允许修复的任务，应单列，不能悄悄从统计里消失。这个指标证明的是修复机制对某类失败的恢复能力，不代表整个系统成功率，也不代表越多修复越好。报告里还应说明修复轮数上限和错误分布，否则两个恢复率可能根本不具有可比性。

P1 33. 要证明 Repair 真有价值，至少要保留哪些“修复成功/仍失败”的真实案例？[待补真实案例]

我会保留首次输入和输出、具体错误、修复反馈、每轮变化以及最终验证结果，至少展示两条成功恢复和一条仍然失败的例子。失败的那条很重要，它能说明系统在哪个边界选择停止，而不是无限尝试。就当前材料，我没有可以确认后直接拿来讲的“三条真实模型 Repair 案例”，所以不会编。实际排查过的 Windows 锁状态误报和测试夹具缺少对账回调，是人工工程修复经历；它们能展示排障能力，但不能冒充 LLM Repair 的实测成果。

端到端任务成功

P0 34. AtlasSplit 的 Task Success 为什么定义为 Plan Valid + Execution Success + Output Verify + Business Constraints Pass?

因为一个任务从理解到交付，任何一层出错都可能导致用户拿到错误结果。Plan Valid 表示计划能被接受，Execution Success 表示执行过程完成，Output Verify 表示产物可读取且符合结构，Business Constraints Pass 才表示它满足业务要求。这四项是逻辑上的“同时成立”，不是把四个分数相加。比如总额分对了但对象选错，依然失败；文件写出来了但不可读，也失败。我希望成功这个状态对应用户真正可以信任的结果，而不只是某个函数正常返回。

P0 35. Plan Valid、Execution Success、Output Verify、Business Constraints Pass 各自是什么意思？

Plan Valid 首先检查结构、必需字段和当前支持的语义，再结合真实输入确认列和数据条件；Execution Success 看执行是否在限制内完成；Output Verify 看文件存在、格式可读、必要字段和行结构是否正确；Business Constraints Pass 则检查总额、对象、比例、精度等业务条件。在项目里，执行结果有 expected_total、allocated_total、difference 和 valid 等机器可读字段。但这些字段本身不意味着所有业务语义已经被证明，仍要看相应检查究竟覆盖了哪些规则。

P0 36. 为什么“代码执行没有报错”远远不等于任务成功？

因为程序可以非常顺利地做错事。比如把“销售额”误认成“利润”，两列都是数字，加权计算不会抛异常，输出也能正常保存。还有一种情况是金额四舍五入后出现尾差，文件没坏，业务上却不守恒。所以我把“执行成功”和“结果正确”拆开，先确认产物，再核验约束。这个思路也延伸到了清理和日志：删除函数没有报错，不一定真的删干净；日志调用成功，也不自动代表完整业务事务已经提交。

P1 37. 首次成功率和 Repair 后最终成功率应该如何放在同一张评测报告里比较？[待补实测]

我会在同一套固定任务上，用相同分母同时报告首次成功数和最终成功数，并把修复带来的新增成功单单独列出来。同时记录多少任务进入修复、修了几轮、额外花了多少时间和调用。对于基础设施异常和取消，提前确定统计规则并保留分类，避免只挑容易成功的样本。当前我还没有这张正式实测报告，所以面试时会说明这是我的评估方案，而不是展示一组估算数字，更不会把不同批次的测试结果拼在一起比较。

安全指标

P0 38. Dangerous Code Detection Recall 是什么？为什么安全场景更怕 False Negative？

危险代码召回率可以理解为：所有标注为危险的样本里，有多少被正确识别并拒绝，也就是 $TP / (TP + FN)$ 。安全场景更怕漏报，是因为一次放过可能带来文件泄露或越权执行，而不是只让一个任务体验不好。不过这个指标必须说明测的是 AST 检测还是整个防御链路。某段代码没有被 AST 拦住，但后来被容器权限阻止，对 AST 来说仍可能是漏报，不能因为最终没出事就把静态检测记成成功。

P0 39. 为什么还要同时看 False Positive Rate，避免安全策略把正常任务全误杀？

如果只看召回率，把所有代码都拒绝就能得到一个很好看的结果，但这显然不是可用的软件。误报率通常是在正常样本里，被错误拒绝的比例，即 $FP / (FP + TN)$ 。对 AtlasSplit 来说，合法的 Pandas 操作是否被白名单误杀，直接关系到产品能完成多少任务。我会把误报样例拿出来分析具体原因，而不是只说策略比较严格。安全和可用性要一起说明，不能用“为了安全”掩盖规则设计上的粗糙。

P0 40. 安全对抗 case 应该覆盖哪些类别，怎样构造才可复现？

我会覆盖危险导入、命令和进程调用、网络访问、越界文件读写、动态执行或反射，以及资源消耗异常。每条样本固定代码、输入、预期判定和规则版本，区分它是在测静态检查还是运行期隔离。运行期样本只放在受控的临时目录和隔离环境，不访问真实敏感文件或真实外部目标。项目里发生过测试目录隔离配置写错的事情，让我更重视这一点：测试安全机制时，测试夹具本身也必须证明不会碰到用户的真实数据。

P0 41. 为什么不能写“100% 安全”，更可信的简历表达应该是什么结构？

我更愿意写成“为模型生成代码增加 AST 白名单审计和 Docker 运行期限限制，并通过具体测试验证若干风险边界”。如果有正式样本集，再补充样本量、风险类别、召回和误报结果，没有就不写百分比。“百分之百安全”既没有清楚的威胁范围，也忽略了宿主配置、依赖和未知行为。当前项目的价值在于把执行边界做得可解释、可检查，而不是宣称消灭所有风险。面试被追问限制时，我也能坦诚说明 subprocess 后端并非完整隔离。

P1 42. 正常任务与安全 case 的分母、Ground Truth、FP/FN 样例应该怎样保存？[待补安全 Benchmark]

正常业务集和安全集应分别有固定 ID、版本和样本清单。业务样本保存输入摘要、需求、预期计划语义和确定性验收条件；安全样本保存危险或正常标签、风险类别、预期拦截层和理由。FP/FN 记录需要链接到原样本和当时的规则版本，避免后面规则改了就找不到当时为什么判错。涉及敏感内容时使用脱敏或专门构造的数据，而不是原样复制用户文件。这里说的是可复现的记录方式，不代表当前已经完成正式安全 Benchmark。

审计与模型比较

P0 43. Audit Replay 有什么价值？如何用于 Debug、事故追踪、回归评测和失败案例分析？

Audit Replay 最有价值的地方，是把“系统当时为什么这么做”从猜测变成可以沿着记录追查。比如用户说自己没有批准当前方案，就可以核对审批摘要、源文件哈希和历史决策；重启后出现重复执行疑问，也能核对 effect ID 和执行代次。当前新增的审计链能发现记录修改、顺序异常和相对本地头锚点的截断，但它不是数字签名；日志和头锚点一起被替换仍可能无法识别。审计验证也不等于完整重放，自动重跑还需要可信输入、环境和授权。

P0 44. Planning、Code Generation、AST Audit、Execution、Output 分别需要记录哪些信息？

规划阶段要关联任务、输入摘要、计划版本和校验结果；代码阶段要记录代码版本或摘要及生成上下文；AST 阶段记录策略和拒绝原因；执行阶段记录后端、执行代次、时限、退出状态；输出阶段记录产物摘要和业务验证结论。实际实现时我会控制隐私，不把凭据和完整表格都塞进日志。项目还有记忆注入收据，记录实际使用的事实 ID、版本、顺序和评分，不记录明文内容。它能帮助定位输入上下文是否变化，但不能据此还原已经删除的记忆全文。

P0 45. 如果换模型，怎样固定 Dataset、Prompt、Tools、Temperature 和 Eval 规则做公平比较？

我会把数据、任务说明、系统提示、工具定义和版本、采样参数、重试预算、验收规则都固定下来，只改变被比较的模型，并记录模型的具体版本和运行时间。缓存、并发和供应商故障也要有统一处理方式，不然会把环境差异算到模型头上。Temperature 相同不代表两个模型行为完全可比，所以还要保留逐题结果，必要时做重复运行。当前没有已经完成的公平模型排名，我不会因为某次演示更顺，就下结论说某个模型全面更好。

P1 46. 模型比较应该同时看哪些指标：Task Success、First-pass、Retry Rate、Latency、Token/Cost？

我会先看最终能不能把任务做对，再看第一次做对的比例、需要多少修复，以及用户等了多久、花了多少调用成本。延迟最好区分活跃处理时间和人工等待，Token/Cost 也要说明来源和是否完整，不能把未知显示成零。项目里预算治理的一个取舍，就是在调用前先记账，结果不明也不随便退款，这能避免重启后重复获得额度。它是费用风险控制，不等同于精确账单；最终账单仍需要可靠的供应商用量数据。

Excel 与 CSV 工程边界

P0 47. Excel / CSV 为什么不能只测试 happy path?

因为用户给的 Excel 和 CSV 往往不是规范数据库导出：列名可能重复，编号带前导零，金额列混着文本，工作表里还有合并单元格和公式。只测一张干净的小表，很容易得到“功能已经完成”的错觉。我的处理思路是先做数据画像和明确校验，把无法可靠解释的情况在执行前暴露出来。前端字段检查也限制读取规模，并在检查前后核对文件摘要，避免用户看到的列信息对应一份文件，真正执行的却已经是另一份。

P0 48. UTF-8/BOM、类型推断、空值、日期、公式、合并单元格、大文件内存这些边界应该怎样分类准备？

我会分成四层。第一层是文件和编码，比如 UTF-8、BOM、扩展名和实际内容；第二层是数据语义，比如空值、混合类型、日期和带前导零的编号；第三层是工作簿结构，比如公式、合并单元格、多工作表和重复表头；第四层是资源边界，比如文件大小、展开后的内存和超时。每层都有明确的接受、拒绝或澄清策略，而不是统一“读失败就换个参数再试”。目前也不能把这些类别一口气说成已经全部做过完整兼容验证。

P1 49. 遇到公式、合并单元格或类型推断异常时，系统应该拒绝、澄清还是降级处理？

我会先看能不能在不改变业务含义的前提下可靠处理。公式要区分表达式和缓存结果，不能假设读取库会像 Excel 一样自动重新计算；合并单元格不能随意向下填充，因为可能把标题当成数据；类型推断异常也不能直接把无法解析的值变成零。涉及业务选择的就澄清，格式不受支持且无法可靠处理的就拒绝，只有规则明确、影响可解释时才降级。降级意味着选择了一个受支持的处理方式，不是为了让任务显示成功而偷偷改数据。

P1 50. 大文件为什么需要关注内存、读取方式和执行超时，而不仅是 Pandas 逻辑正确性？

因为文件在磁盘上小，不代表展开到内存以后也小，Excel 解压、DataFrame 复制和中间聚合都可能放大占用。算法逻辑正确，但把服务内存耗尽，一样是失败。项目已做文件大小、输入检查子进程、执行资源和超时等边界，也有低磁盘准入；但这些不等于已经支持任意大文件流式处理。对更大规模的数据，我会先量清楚瓶颈，再决定是否分块或换处理方案，不会只把 Pandas 代码写对就宣称具备大数据处理能力。

被追问项目经历时怎么展开

面试官如果问最有价值的一次排障，我会讲审批竞态：前端看过计划，不意味着后端收到批准时还是同一份上下文。后来把源文件、计划、配置、策略和有效期绑定起来，批准时做摘要比较，执行前再检查。这个问题让我把“按钮上有确认”与“服务端真的建立了授权”区分开了。

另一个很具体的经历是 Windows 文件锁。清理流程需要持有任务锁直到删除完成，但锁文件原来就在待删除目录里，Windows 上会和删除冲突。把锁位置移到目录外之后，检查又暴露了锁文件释放时自动删除造成的状态判断竞态，所以不能先 `exists` 再 `is_file`，而要减少多次读取之间的窗口。这类问题单看业务代码很难想到，必须通过实际并发检查来发现。

还有一次教训是测试配置用了错误的字段名，导致本以为写进临时目录的数据落到了仓库默认运行目录。后续不是直接删掉当作没发生，而是先列清误写范围，保留审计和现场，并把测试启动前的运行目录断言补上。真实数据处置仍需要单独授权。这件事让我意识到，安全不只存在于产品代码里，也存在于测试和开发工具的默认行为里。

如果问项目还有什么没做完，我会明确说：当前不是全能表格 Agent，也没有完成生产发布证明。TablePlan 的完整 HTTP 入口、部分恢复和回放能力仍有待办；正式业务成功率和安全 FP/FN 数据也不能凭工程测试数量推导。这个项目更适合展示我如何把一个能跑的 LLM 原型，逐步变成有明确状态、权限边界、失败处理和可追踪证据的软件。

实现核对线索

计划字段与支持范围： `planning/plan_models.py`、`planning/table_models.py`。

静态审计与后端边界： `execution/audit.py`、`sandbox/sandbox.py`。

审批与执行权： `server/approval_authority.py`、`server/dispatch.py`、`server/jobs.py`。

审计完整性： `core/audit_integrity.py`、`core/audit_log.py`、`cli_audit.py`。

记忆与清理经历： `core/memory_receipt.py`、`server/retention.py`，以及任务检查点和测试隔离事件记录。

Agent 技术扩展面试问答

新增 40 道追问 · 题号 51 至 90 · 第一人称口述稿

从记忆如何进入规划，到任务怎样暂停、恢复和取得执行权限。本节继续区分实现事实与设计边界，尤其不将 FTS 检索说成向量检索、不将开发协作说成多 Agent 产品，也不将局部幂等说成 exactly-once。

Agent 架构与技术选择

51. 你这个项目到底用了哪些 Agent 技术，能先整体讲一下吗？

我会沿着一次任务来讲：用户提出需求后，系统先看数据画像，必要时检索历史经验；模型可以调用有限的检查和预览工具，生成计划，遇到歧义就停下来问。之后通过持久化流程等待审批，获得有效授权才执行，结果再由确定性规则验证。任务成功后，值得保留的经验可以异步进入候选记忆。这里用到了工具调用、短期与长期记忆、LangGraph 暂停恢复、人工介入，以及预算和审计。我的重点不是让 Agent 尽可能自由，而是让每一步的输入、权限和失败处理都说得清楚。

52. 这和普通的 LLM 接口套壳有什么区别？

最明显的区别是，系统不是拿到模型回答就结束。模型可以根据工具反馈继续规划，系统也会把任务状态持久化，在澄清和审批处暂停，重启后继续处理，并核验最终文件。但是我也不会把它包装成通用自主智能体。AtlasSplit 是一个业务范围有限的表格 Agent，常规分摊的计算和执行权限仍由确定性代码控制。我觉得它的工程价值，在于把“模型建议怎么做”和“软件最终允许做什么”分开，而不只是多接了几个模型接口。

53. 这是 ReAct、Workflow，还是多 Agent 系统？

我会说它是有界工具调用加确定性工作流。工具循环有“检查数据、拿反馈、调整计划”的特征，但没有必要因此宣称实现了某个完整 ReAct 范式，更不会记录或展示模型内部思维链。LangGraph 负责暂停与恢复等控制流程，主运行路径也不是多个自主 Agent 相互协商。开发过程中用多个 AI 助手协作，与软件本身采用多 Agent 架构是两回事。面试时我会明确区分，不把开发工具的能力算成产品能力。

54. 为什么没有直接拆成规划 Agent、执行 Agent、审查 Agent？

我考虑的首先是这些角色是否真的需要独立决策。执行金额分摊和检查总额守恒，用确定性函数通常比再叫一个模型更可靠。如果把每一步都包装成 Agent，会增加调用成本、状态同步和冲突处理，却未必增加业务价值。目前我选择模块分工，而不是多个模型角色互相确认。以后只有出现可以独立完成、确实需要不同上下文或工具权限的子任务，才有理由引入多 Agent；不能为了架构图看起来丰富，就先增加协作复杂度。

55. 你说有记忆，具体分哪几层？

当前主要分会话记忆和长期事实记忆。会话层保存 CLI 聊天中的用户与助手轮次，并在上下文变长时压缩较早内容；长期层把列映射、偏好或操作经验等提炼成 SQLite 中的事实，下一次规划前检索。除此之外还有任务状态和 checkpoint，但我不会把它们都叫长期记忆：它们记录的是流程进行到哪里、是否具备执行权限。历史经验可以帮助理解，任务账本则决定能不能继续执行，这两种信息的责任不能混在一起。

56. 为什么不直接把所有历史聊天都塞给模型？

因为历史越多不一定越有用。完整聊天会增加成本，挤占当前输入的空间，还可能把已经过时的要求或敏感信息一起带进去。我做的是保留近期轮次，把较早内容压缩成摘要，再从长期库里少量召回相关事实。不过摘要也会损失细节，所以我不会把它当成原始证据。涉及当前任务的列选择、输入文件和审批，仍要以这次明确提供并持久化的内容为准，不能靠摘要里的印象直接作决定。

57. 会话摘要失败了怎么办，会不会卡住用户？

摘要压缩放在后台做，调用有边界，失败时保留受限的文本摘录作为降级，不让聊天一直等着摘要服务恢复。这个选择有代价：降级摘要可能不如模型整理得清楚，也不能保证历史细节完整保留。所以我的目标是维持当前交互可用，而不是声称压缩完全无损。会话日志和有效上下文也要区分，写到了历史文件里，不代表下一轮会把它全部发给模型；进入模型的内容仍受上下文预算限制。

58. 长期记忆保存什么，为什么不保存整张 Excel？

我希望长期保存的是可复用的经验，而不是把业务原始数据再复制一遍。比如某种列名通常对应什么含义、用户明确确认过的偏好、某类任务的处理经验，才值得考虑进入长期库。整张 Excel 既容易过期，也有隐私和容量问题。当前提取路径会约束输出并做脱敏，但脱敏规则不可能证明所有敏感内容都被识别了。因此我会把“避免存原始文件”作为设计边界，不会说模型提炼过一次就天然没有隐私风险。

59. 记忆是怎么写进去的，是任务结束就同步调用模型吗？

不是让已经成功的任务再等一次模型调用。API 的成功任务会进入受控的后处理流程，把记忆提取任务持久化到队列，后台领取后再做有限提取。这样记忆服务暂时不可用，不会把已经成功的分摊结果改成失败。这里我特别在意先落盘再确认，否则进程刚好退出，内存里的后处理任务就丢了。当前可靠性提升主要在这个持久化队列和领取协议上，而不是简单开一个线程然后假设它一定会做完。

60. 队列失败重试时，会不会重复调用模型、重复写记忆？

有这个风险，所以队列会使用任务身份和租约，并把提取结果持久化后再确认完成。这样如果结果已经落盘，只是后续写入或确认失败，恢复时就能复用结果，不必再生成一次。但如果进程在模型已经返回、结果还没落盘的窗口崩溃，仍可能重复提取。我不会把这个实现叫 exactly-once。更准确的说法是，通过持久化结果和去重减少重复，并确保保留无法彻底消除的不确定窗口。

61. 模型自动提取的记忆可信吗，为什么还要候选和确认状态？

模型提取的内容可能理解错，也可能把一次偶然情况总结成固定规则，所以我让它先作为候选事实存在，而不是一写入就视为真相。用户可以确认、拒绝，或者用新事实替代旧事实。当前检索也可能使用有效候选，但会标明候选身份，并优先排列已确认内容。这里的确认只是提高了这条经验的可信程度，不是授予它执行权限；即使记忆已经确认，仍不能覆盖这次用户的明确要求和系统安全规则。

62. 两条记忆互相矛盾时怎么办？

我不想用“后写入的自动获胜”处理所有冲突，因为后来的内容也可能更不可靠。当前治理接口支持确认、拒绝和替代，同一个命名空间及键下确认新候选时，可以保留它与旧事实的替代关系。运行诊断还会显示潜在冲突组，方便检查。不过这只是基于结构化身份和内容差异的冲突线索，不是一个能理解任意自然语言矛盾的推理系统。真正涉及业务含义的冲突，仍需要明确选择，不能靠时间戳替用户决定。

63. 记忆为什么需要过期？用户确认过的也会过期吗？

因为列映射和操作经验都可能随着文件和业务变化而失效。当前候选记忆默认按创建时间计算三十天有效期，不会因为被频繁命中就不断续命；有显式有效期时按显式规则处理。确认后的事实不再受这个默认候选期限限制，但仍要遵守它自己的有效期。这里做的是检索时过滤，不是到期就物理删除。这样既减少旧经验误导模型，也保留后续排查和人工治理需要的记录。

64. 你的记忆检索是向量检索吗，为什么用 SQLite FTS5？

当前是 SQLite FTS5 文本检索，不是向量库。对这个本地单用户项目，记忆主要是短事实、列名和操作经验，先用已有 SQLite 管理数据和检索，依赖少，也容易观察为什么命中。短中文查询还需要考虑字面匹配的退路。不过 FTS 对同义表达和语义改写的理解有限，这个限制我会承认。以后如果真实查询证明词面匹配成为主要瓶颈，再考虑向量或混合检索，而不是先增加嵌入服务和索引维护成本。

65. 召回后怎么排序？confidence 越高就一定越对吗？

当前先按信任层区分已确认和候选，再结合文本相关性、confidence、封顶的证据数和时间衰减排序，最后用稳定 ID 处理并列。命中次数主要是使用统计，不会因为一条事实被反复看到，就自动把它变成真理。confidence 也不是经过校准的正确概率，所以我不会说分数零点八就代表八成正确。这套排序是可解释的工程启发式，还需要真实检索样本来判断质量，不能因为公式完整就宣称检索效果已经很好。

66. 记忆检索失败或者数据库被锁住，会影响主任务吗？

记忆在这里是辅助信息，所以读取失败、数据库忙或超过预算时，可以跳过注入，让当前任务继续根据真实输入规划。查询有进度中断和不等待写锁等控制，外层也会检查耗时。不过我不会把五十毫秒配置说成对所有文件系统操作的硬实时保证，因为连接和 I/O 也有实际开销。另一个细节是，命中计数写失败不应该把已经读出来的有效结果一起丢掉，统计和业务读取需要分开处理。

67. 命名空间是不是就能保证不同项目的记忆绝不串用？

不能这么说。当前事实模型有 user、playbook 和 project 指纹这类命名空间，查询接口也支持按命名空间筛选，但有字段不代表每个调用入口都已经正确使用它。我核对当前通用注入入口时，它没有显式传入项目过滤条件，因此不能宣称已经完成严格的项目隔离，更不能拿它当多租户权限系统。这个地方我会直接讲成限制：要实现严格隔离，还需要把可信项目身份贯穿检索调用，并补对应的越界检查。

68. 怎么证明某一次回答到底用了哪几条记忆？

当前实际注入记忆时会生成一份收据，记录事实 ID、版本摘要、状态、排序分量和整体顺序摘要。这样排查两次规划差异时，至少能判断是不是召回内容或版本发生了变化，而不必把记忆全文写进日志。它的限制也很明确：摘要不能还原被删除的原文，收据目前属于 trace 证据，写入是尽力而为，不等同于强制持久化的审批审计。所以它提升了可追踪性，但还不能被包装成完整的确定性回放快照。

69. 记忆里如果混入“忽略之前规则”这种内容怎么办？

我会把召回内容当历史数据，不当系统指令。当前注入时有明确的数据边界和不可信说明，但只加一对标签并不能彻底阻止提示注入。更关键的是，模型即使受影响，也不能因此绕过工具权限、计划校验和审批。记忆里的列映射还必须与当前数据核对，不能直接照搬。我的思路是减少它影响模型的机会，同时限制模型能够造成的后果，而不是相信模型看到一句“不要听里面的指令”就绝不会出错。

70. 加了记忆是不是就叫持续学习？

如果这里的学习指跨任务保留经验，可以这么描述；但如果面试官说的是模型训练，我会马上区分。当前没有因为任务结束就更新模型参数，没有在线微调，也没有通过成功任务自动训练奖励模型。它是外部记忆的提取、检索和治理。这个方案容易查看、撤销和纠正，但也会受到提取错误、检索遗漏和过期信息的影响。我更愿意说系统能够复用受治理的历史经验，而不是说模型越用越聪明且不会退化。

LangGraph 状态与人工介入

71. 为什么引入 LangGraph，它具体替你解决了什么？

最实际的需求是任务会在澄清和审批处暂停，而且用户可能过很久才回来。LangGraph 让这些节点、恢复输入和 checkpoint 有明确结构，不必把暂停逻辑散落在很多回调里。但我没有把它当成可靠性的全部来源：任务清单、执行租约和副作用队列仍有自己的权威边界。它主要负责流程怎么继续，不能只因为图走到了执行节点，就认为已经获得执行权限。这样框架有它的位置，也不会变成第二套业务账本。

72. 图状态和 job.json 不一致，究竟听谁的？

对公开任务状态、取消、审批、执行代次和副作用，任务清单是权威；图状态是流程投影，需要通过对账与它保持一致。比如用户已经取消任务，即使 checkpoint 还停在旧节点，也不能据此继续执行。之所以这样设计，是为了避免恢复时两边都说自己是最新状态。我会让真正产生副作用的入口重新核验清单和租约，而不是相信某个缓存页面或图节点的历史结论。单一权威比“所有地方都有一份状态”更重要。

73. LangGraph interrupt 有什么容易踩的坑？

一个很重要的点是，恢复时被中断的节点可能从头重新执行，而不是简单从那一行往后接着跑。如果在 interrupt 前发了消息、写了结果或启动了进程，恢复时就可能重复发生。所以这个项目把暂停节点尽量写成不产生外部副作用的控制逻辑，真正的执行通过持久化意图和协调器管理。这个经验让我更关注恢复语义：代码顺序看起来只执行一次，不代表框架在重试和恢复条件下也只会执行一次。

74. checkpoint 能恢复，是不是就能直接反序列化任何旧状态？

不能。checkpoint 是持久化输入，也要防止损坏、版本漂移和不安全对象恢复。当前有受限序列化、载荷边界以及工作流版本和指纹检查，不能只设置一个环境变量就认为反序列化安全已经解决。遇到不兼容记录，应该停止自动执行并保留原数据，不是为了让流程跑起来就自动删掉重建。因为旧状态里可能关联用户审批和执行意图，失去这些上下文以后再运行，反而可能制造更严重的问题。

75. 澄清和审批为什么要分开，不都是问用户一句吗？

澄清解决的是“你到底希望怎么做”，审批解决的是“你是否允许执行这份已经明确的方案”。用户回答“用销售额列”，并不等于允许马上修改文件。澄清还需要绑定问题 ID 和当前轮次，旧页面提交的答案不能套到新问题上；审批则需要绑定计划、输入和有效上下文。界面上它们都可能是一个按钮，但在协议上是不同的状态转移。把两者合并成一个模糊的确认，会让恢复和权限判断都很难解释。

76. 用户点过批准，为什么执行前还要再检查？

批准和执行之间可能隔了一段排队时间，输入可能变化，审批也可能过期。项目里专门补了这个窗口：批准时比较用户看到的上下文摘要，执行领取和启动前再检查是否仍有效。失效时要把撤销租约、停止派发和回到待审批的状态落盘，再向调用方报冲突；否则只抛异常，状态更新可能根本没有保存。这不是让用户反复确认同一件事，而是保证真正执行的仍然是他批准的那件事。

77. 旧任务没有这些审批字段，升级后怎么处理？

我没有给历史记录凭空补一个看起来有效的审批时间。新任务显式带保护标记，批准时必须提交当前摘要；真正没有上下文的旧记录保留明确的兼容路径，但不能被说成同样具备过期保护。重新审批还要保留旧决策，因为失败后重试的审计投影可能仍引用旧的 effect ID。这里最容易犯的错是把旧审批直接清空，再用新审批代替它，结果历史记录就被悄悄改写了。

工具权限与调用控制

78. Function Calling 里模型能决定用什么工具，权限会不会失控？

模型能提出调用请求，但工具名必须来自注册表，参数要校验，状态和副作用策略由服务端决定。当前规划工具主要是 workbook_summary、preview_plan 和提交计划，歧义通过明确的澄清协议暂停。工具的效果类别、是否需要审批、允许状态和输出限制是可信运行时元数据，不是让模型在参数里自报。我不会给规划循环一个通用写文件工具，再指望提示词约束它；需要持久化副作用的操作应该走执行协调器。

79. 工具参数过了 JSON Schema 就安全吗？

还不够。Schema 能约束结构，但列名是否存在、路径是否在允许目录里、权重是否有效，都要结合真实数据和运行上下文检查。像 generation、lease_id 和剩余时限，也不能接受模型自己传一个值就信任，而应从实际调用环境获得。项目里的工具注册还会检查封闭对象结构、处理函数和有限的时限配置。我把这些理解成不同层次的校验：格式正确只是入口条件，不等于拥有执行权限或满足业务语义。

80. 工具循环怎么停下来？模型一直想再试一次怎么办？

循环既有轮数上限，也要共享当前规划的调用预算和截止时间，不能每进入一个子循环就重新获得额度。协议错误有时允许在同一预算里退回单轮规划，但处理函数失败、取消、澄清和预算耗尽不能被一个宽泛异常捕获后当作“再试一次”。还有一个实际限制：普通同步 Python 函数的超时检查通常是协作式的，不能保证把任意卡死函数强行终止。需要硬执行边界的检查，应放到已有的隔离子进程里。

预算 恢复与副作用

81. 重试、熔断和预算分别解决什么，为什么三个都要有？

重试是为了给暂时性故障一次恢复机会，熔断是在供应商持续失败时减少无效请求，预算则约束单个任务最多花多少调用。它们不能互相替代：有重试上限，很多任务同时重试仍可能压垮服务；有熔断，也不能允许一个任务在正常服务上无限消耗。当前运行时由应用持有，活跃规划轮次绑定范围，嵌套调用复用它，而不是隐藏在模块全局变量里。后台线程也要显式建立自己的范围，不能假设线程上下文会自动继承。

82. 进程重启后，为什么不能把 LLM 调用预算重新从零开始？

因为重启不是新的业务授权。如果任务已经调用了好几次模型，崩溃恢复后额度又满了，就能通过反复重启绕过限制。当前 API 和 CLI 会在 SDK 调用前持久化预留次数，结果不明确时不随便返还。旧记录如果没有可靠用量，就不能从诊断日志猜出一个精确值，再给它新的调用额度；但读取历史结果和已有审批允许的确定性执行可以另行判断。这个取舍偏保守，目的是不把系统的不确定性变成用户的额外账单。

83. 多个 worker 抢同一个任务，怎么防止旧 worker 把结果写回来？

任务领取除了互斥，还会使用租约和执行代次。租约说明当前谁在处理，代次用于识别恢复以后已经过期的执行者。旧 worker 即使晚一点返回，也要在提交结果和副作用前重新检查自己是否仍有权限。这里不能只看“任务处于执行中”，因为新旧两次执行都可能对应这个状态。这个机制可以减少旧结果覆盖新状态，但对真正外部系统的副作用，仍需要对方支持幂等或独立核对，不能只靠本地字段保证绝不重复。

84. 为什么要做 outbox，直接执行完再写审计和记忆不行吗？

直接顺序调用会留下一个窗口：业务结果已经成功，进程却在写审计或记忆前退出，之后就不知道还有什么没做。outbox 把状态变化和待处理副作用放进同一次持久化更新，再由后续处理器按 effect ID 完成投影。失败时可以继续处理，不必重做业务计算。但它也不是数据库之间的魔法事务，所以每个处理器仍要考虑重复调用。清理流程也不能只看任务成功，就删除还有待处理或失败副作用的目录。

85. 用户取消后后台还在运行，是不是取消功能有 bug？

要先区分取消被接受和执行进程已经退出。系统应该及时撤销继续执行和提交结果的权限，并把取消状态持久化，但线程、子进程和外部请求停下来的时间可能不同。界面不能把“已提交取消”说成所有资源都已释放，晚到结果也不能把取消状态改回成功。后台清理和对账要继续跟进。这也是我不喜欢只用一个 cancelled 布尔值覆盖所有阶段的原因：用户意图、执行权和实际进程生命周期需要关联，但不是同一件事。

86. 后台记忆或对账线程死了，怎么发现、怎么恢复？

后台循环交给监督器管理，区分到期调度、暂时失败和需要明确处理的降级状态。像短暂连接故障可以回避重试，TypeError 或 AssertionError 这类程序错误不该无限自愈。诊断接口要让人看得到哪个循环不可用，而不是服务还能打开页面就显示一切正常。当前对部分循环支持显式重试或恢复，但这不等于完整高可用集群；本地进程重启、持久化队列恢复和人工处理仍各有边界。

可观测性与面试总结

87. trace、审计日志和任务状态为什么不能合并成一份？

它们回答的问题不同。任务状态决定现在能做什么，审计记录关键决策和副作用身份，trace 帮助解释某次调用和失败的过程。如果把执行授权依赖在可能丢失的诊断日志上，日志写失败就会变成业务状态混乱。反过来，把所有调试细节塞进权威清单，也会让它越来越重。当前记忆注入收据是 trace，审批历史是决策证据；这种区分能让我明确哪些信息可以尽力写入，哪些必须先持久化才能继续。

88. 怎么观察 Agent 有没有失控，又不泄露用户数据？

我会关注调用次数、状态转移、重试原因、预算耗尽、工具耗时和验证失败这些可操作信息，用任务 ID 和请求 ID 关联，而不是记录整张表格或凭据。当前也有只读诊断和记忆治理计数，读取失败显示未知，不伪装成零。脱敏不是万能的，所以还要控制记录内容本身和导出范围。我不会为了调试方便去保存模型隐藏思维过程，也不会把摘要哈希说成匿名化的绝对保证；它主要用于关联和完整性核对。

89. 如果面试官问记忆和 Agent 机制到底带来了多少提升，你怎么回答？

我会先说能确认的工程收益：任务可以在审批或澄清处持久化暂停，预算不会因为普通重启自动重置，记忆提取失败不需要推翻已经成功的结果，历史上下文也更容易追踪。但这些不能直接转成“准确率提升百分之多少”。要证明记忆效果，需要固定任务，比较开启和关闭记忆时的检索命中、任务结果、额外延迟和错误注入情况。目前没有这组正式对照数字，我会坦诚说明，不用功能数量替代效果证据。

90. 如果只能挑一个亮点讲两分钟，你会怎么讲？

我会讲从“模型能生成计划”到“系统能可靠执行计划”的变化。最开始看似只要规划、执行、输出，但加入人工等待和失败恢复以后，就会遇到旧审批、重复调用、晚到结果和后处理丢失这些问题。后来我把流程控制、任务权威状态和历史记忆分开：LangGraph 管暂停恢复，清单与租约管执行权，记忆只提供受治理的参考。最具体的一次改进是给审批绑定摘要并在执行前复查。这让我学到，Agent 工程最难的往往不是让模型多做一步，而是确保失败或重启以后，它不会做错那一步。

数据不足与能力边界怎么回答

新增 13 道直接追问 · 题号 91 至 103

目前主要的短板是效果证据不足和部分能力尚未闭环，并非已经测出了很差的百分比。下面的回答先回应问题，再说明现有证据、真实限制和下一步，不用技术名词掩盖不足，也不把合理取舍说成失败。

91. 没有正式任务成功率，怎么证明你这个项目真的有用？

这个问题我会分两层回答。业务效果上，我目前还没有统一固定任务集上的成功率，所以不会拿单元测试通过数代替准确率。工程行为上，我能展示一个任务从数据检查、计划预览、审批到执行验证的完整过程，也能解释审批失效、重启和取消时系统怎么处理。它证明的是这个流程可以被检查、被控制，不是已经证明能处理大多数真实业务。下一步要补的是固定任务和独立验收条件，把具体案例变成有统一口径的数据，而不是先给简历填一个百分比。

92. 你做了这么多记忆功能，但怎么知道它没有越帮越忙？

我不会默认记忆一定有正收益。历史经验可能帮助识别列名，也可能把上一次的错误带到这一次，所以它必须可以关闭，并且不能覆盖当前输入和审批。当前已经能记录实际注入的事实身份、版本和顺序，至少让问题可追踪；但还没有正式的有无记忆对照结果。真正要证明价值，我会比较任务结果、错误经验干扰和额外耗时。如果发现某类记忆经常误导，就缩小注入范围或停用，而不是因为已经做了这个功能，就必须证明它有用。

93. 没有真实 Repair 恢复率，你为什么还把 Repair 当项目特点？

如果把它写成已经显著提升成功率，那确实不合适。我目前更准确的表述是，系统有受限制的修复和重试流程，并对错误分类、预算和停止条件做了治理。至于修复能救回多少任务，还需要真实样本证明。我也不会把开发过程中人工修复文件锁问题，算成模型自动修复成功。面试时可以讲这个机制为什么需要、怎样避免失控，但在效果数据出来之前，不把它当成已经验证的性能亮点。

94. 主路径只有等额和权重分摊，会不会业务太简单？

业务规则本身确实是有意收窄的，我不会把这两类计算说成算法创新。这个项目更有挑战的部分，是用用户用自然语言描述需求以后，系统如何确认列和总额、展示计划、等待审批，并在异常或重启后不执行错方案。TablePlan 已有一些通用操作，但入口闭环还没有全部完成。我宁愿把支持范围讲小一点，再把完整流程展示出来，也不写“六类规则”然后拿不出逐类实现。后续是否扩展规则，应该由真实需求决定，而不是为了让功能列表更长。

95. 你既没有安全召回率，也不能保证容器绝对安全，安全部分还有什么可讲？

可以讲的是具体防御措施和它们各自的边界，而不是一个没有依据的安全结论。当前有 AST 白名单、危险调用检查，以及 Docker 后端的网络配置、只读文件系统和资源限制。我会明确说明哪些配置已经在代码里，哪些部署加固还需要补齐，也会区分 Docker 和本地子进程后端。没有正式召回率和误报率，就不报数字。但这不意味着控制措施毫无价值，只是它们目前证明了某些风险受到约束，还没有证明覆盖了所有攻击方式。

96. 你有 namespace 却没有贯穿项目过滤，这是不是一个实际缺陷？

是，如果要求不同项目的记忆严格隔离，那当前通用注入入口没有显式传入项目过滤条件，就是需要补齐的缺口，不能只说有 namespace 字段就算完成。即使是本地单用户，也可能有不同项目的数据边界。我会先明确当前不能承诺严格隔离，再把可信项目身份传入检索调用，并验证不同项目之间不会误召回。在这之前，不应该把它宣传成多租户能力；对有严格隔离要求的使用场景，也不应该默认开启跨任务记忆。

97. 测试居然写进了默认运行目录，你怎么解释这种失误？

这件事确实是失误，不能用“只是测试数据”轻描淡写地带过。当时配置用了错误的字段名，本以为指定了临时目录，实际仍落在默认运行路径。发现以后先停止相关操作、核对误写范围并保留现场，没有直接删除记录把问题掩盖掉。后续把测试启动前的路径断言补上，要求每个运行目录都确实位于隔离区域内。真实数据处置尚未全部完成，我也不能说已经无损恢复。这个教训是，测试安全不能只依赖配置写对，还要在启动时验证配置真的生效。

98. 为什么记忆还在用 FTS5，不用向量数据库是不是技术不够先进？

我选择 FTS5 主要是因为当前记忆是短事实、列名和操作经验，项目又是本地单用户，先用 SQLite 可以减少依赖和运维，也容易查看命中的原因。这不是说向量检索没价值：遇到同义表达和语义改写，词面检索确实可能漏掉内容。不过换成向量数据库还会带来嵌入成本、索引版本和删除一致性问题。我的判断标准是实际查询有没有暴露主要瓶颈，而不是技术名词是否更新。如果事实证明语义召回不足，再做混合检索会更有依据。

99. 没有多 Agent，你的 Agent 架构是不是不够完整？

我不把 Agent 数量当成完成度。当前任务里，规划需要模型理解，执行和金额验证更适合确定性代码。如果再引入一个审查 Agent，它可能只是用另一段自然语言重复前一个模型的判断，还会增加成本和状态同步问题。所以现在采用有界工具调用和明确工作流，把需要模型决策的地方集中起来。以后出现真正可以独立处理、拥有不同上下文和权限的子任务，再考虑多 Agent。对这个项目来说，少一层没有实际收益的协作，是设计取舍，不是少实现一个标签。

100. 你承认不能保证 exactly-once，那幂等和恢复还有意义吗？

有意义，但要把保证范围讲准确。比如记忆提取结果已经持久化，后续确认失败时可以复用结果，避免再调用一次模型；副作用也可以通过 effect ID 识别重复处理。但如果模型调用成功后、结果落盘前进程崩溃，系统仍可能不知道调用是否完成。遇到这种不确定性，本地幂等不能凭空提供外部事务保证。我的目标是缩小重复窗口、阻止重复提交，并保留可核对的记录，而不是用一个 exactly-once 术语掩盖仍然存在的故障条件。

101. 审计哈希链不是签名，也不能完整重放，是不是名字起得太大了？

如果把当前能力直接叫“不可篡改的完整回放系统”，那确实说大了。当前审计链能检查记录内容、顺序和相对本地头锚点的一致性，帮助发现修改或截断；但日志与锚点一起被替换，并不一定能识别。完整重放还需要当时可信的输入、环境、配置和授权，哈希本身不能恢复这些内容。所以我会把现阶段描述为审计完整性检查和追踪基础，而不是完整回放已经交付。名称应该跟能力走，不能让一个漂亮名称替代实现。

102. 项目还没达到生产发布标准，你为什么认为它值得写进简历？

因为简历项目可以展示解决问题的过程和工程能力，但前提是明确它处在哪个阶段。我会把 AtlasSplit 定位成具有受控执行流程的本地项目，不会写成已经服务大量用户的生产平台。它值得讲的部分是，如何把自然语言规划接到确定性计算，如何处理人工等待、重复调用、状态恢复和审计；不足则是正式业务效果数据、部分入口整合以及生产部署验证还没有完成。这样面试官可以根据真实实现判断，而不是被一个过大的产品定位误导。

103. 这些短板不少，如果用一分钟总结，你到底做成了什么？

我做成的不是一个已经证明能处理任意表格的全能 Agent，而是把有限的分摊任务接成了可预览、可审批、可验证的执行流程，并逐步补上重启、取消、预算和后处理失败这些容易被原型忽略的问题。这个过程中我也发现了一些边界，比如记忆隔离还不完整，效果数据也没有统一口径。我会先展示能实际跑通和核对的主链路，再解释取舍与限制。对我来说，最有价值的收获是知道哪些判断可以交给模型，哪些权限和正确性必须由软件自己守住。